COLE DES HAUTES ETUDES D'INGENIERI

HEI Oujda — Departement Genie Informatique

Filiere : Genie Informatique — Groupe 4GI B

Annee universitaire : 2024–2025

Projet de Fin d'Annee

MedSys

Systeme de Gestion Hospitaliere Intelligente

Architecture Microservices — Spring Boot — React — React Native

Realise par :

- **Douae DEROUICH**
- **Oumayma DARDOUR**
- **Ouissal YAHYAOUI**
- **Badr LAAJI**
- **Mohammed ALOUANI**

Encadre par :

[Nom de l'encadrant]

Soutenu le :

[Date de soutenance]

Dedicace

A nos familles, pour leur soutien indefectible tout au long de ce parcours.

A nos enseignants de l'Ecole des Hautes Etudes d'Ingenierie d'Oujda,
pour la rigueur et la passion qu'ils ont su nous transmettre.

A tous ceux qui croient que la technologie peut ameliorer
la qualite des soins de sante.

Remerciements

Nous tenons a exprimer notre profonde gratitude a toutes les personnes qui ont contribue a la realisation de ce projet de fin d'annee.

Nos premiers remerciements vont a notre encadrant, Monsieur Soufiane DAHMANI, pour ses precieux conseils, sa disponibilite et son suivi rigoureux tout au long de ce projet.

Nous remercions egalement l'ensemble du corps enseignant de l'Ecole des Hautes Etudes d'Ingenierie d'Oujda, et en particulier les enseignants du departement Genie Informatique, pour la qualite de la formation dispensee.

Enfin, nous adressons nos remerciements a nos familles et proches pour leur soutien moral et leurs encouragements constants.

Resume

Ce rapport presente **MedSys**, un systeme de gestion hospitaliere intelligente developpe dans le cadre du Projet de Fin d'Annee (PFA) de la filiere Genie Informatique, 4eme annee, a l'Ecole des Hautes Etudes d'Ingenierie (HEI) d'Oujda.

MedSys est une solution complete de gestion hospitaliere reposant sur une **architecture microservices** developpee avec Spring Boot (Java 21). Le systeme permet la gestion des patients, des dossiers medicaux electroniques, des consultations, des ordonnances, des analyses de laboratoire, des radiologies, des hospitalisations et de l'authentification securisee par JWT.

L'application est accessible via une interface web (React/Vite) et une application mobile cross-platform (React Native/Expo).

Mots-cles : Microservices, Spring Boot, Spring Cloud, Dossier Medical Electronique, JWT, React, React Native, MySQL, Docker.

Abstract

This report presents **MedSys**, an intelligent hospital management system developed as a Final Year Project for the Computer Engineering program (4th year) at HEI Oujda, Morocco.

MedSys is a comprehensive hospital management solution built on a **microservices architecture** using Spring Boot (Java 21). The system handles patient management, electronic medical records, consultations, prescriptions, laboratory analyses, radiology, hospitalizations, and JWT-based authentication.

Keywords : Microservices, Spring Boot, Electronic Medical Record, JWT, React, React Native, MySQL, Docker, Hospital Management.

Table des matières

Dedicace **1**

Remerciements **2**

Resume **3**

Table des matieres **5**

Liste des figures **6**

Liste des tableaux **7**

Liste des abreviations **8**

1 Introduction Generale **9**
1.1 Contexte et Problematique 9
1.2 Objectifs du Projet 9
1.3 Perimetre Fonctionnel 9
1.4 Organisation du Rapport 10

2 Analyse et Specification des Besoins **11**
2.1 Identification des Acteurs 11
2.2 Diagramme de Cas d'Utilisation Global 12
2.3 Besoins Fonctionnels 12
2.3.1 Module Authentification (ms-auth) 12
2.3.2 Module Patient et Dossier Medical (ms-patient-personnel) . . . 12
2.4 Besoins Non Fonctionnels 13

3 Conception **14**
3.1 Architecture Generale 14
3.1.1 Choix Architectural : Microservices 14
3.1.2 Diagramme d'Architecture 14
3.2 Conception UML 15
3.2.1 Diagrammes des cas d'utilisation : 16
3.2.1.1 ms-rendez-vous 16
3.2.2 Diagrammes de classe 17
3.2.2.1 ms-auth 17
3.2.2.2 ms-patient-personnel 17
3.2.2.3 ms-rendez-vous 19
3.2.3 Diagrammes de Sequence 21
3.2.3.1 Sequence : Authentification JWT 21
3.2.3.2 Sequence : Ajout d'une Consultation 22
3.2.3.3 ms-rendez-vous 23
3.2.4 Diagrammes de Deploiement 24

3.3 Conception Merise 24
3.3.1 Modele Conceptuel de Donnees (MCD) 25
3.3.1.1 ———- 25
3.3.1.2 ms- rendez-vous 26
3.3.2 Modele Logique de Donnees (MLD) 26
3.3.3 Modele Physique de Donnees (MPD) 28
3.3.4 Dictionnaire de Donnees 29

4 Realisation et Implementation 31
4.1 Environnement de Developpement 31
4.2 API REST des Microservices 31
4.2.1 ms-auth (port 8082) 31
4.2.2 ms-patient-personnel (port 8081) 32
4.3 Securite : JWT et RBAC 33
4.4 Numerotation Automatique des Dossiers 33
4.5 Interfaces Utilisateur 33
4.5.1 Application Web (medsys-web) 33
4.5.2 Application Mobile (medsys-mobile) 34
4.6 Difficultes Rencontrees et Solutions 34

5 Conclusion et Perspectives 35
5.1 Bilan du Projet 35
5.2 Perspectives d'Amelioration 35
5.3 Conclusion 35

Bibliographie 36

Table des figures

2.1 Diagramme de cas d'utilisation global de MedSys 12

3.1 Architecture generale de MedSys 14
3.2 Diagramme de use case : ms-rendez-vous 16
3.3 Diagramme de classes — ms-auth 17
3.4 Diagramme de classes — ms-patient-personnel 18
3.5 Diagramme de classe : ms-rendez-vous 19
3.6 Diagramme de sequence — Authentification JWT 21
3.7 Diagramme de sequence — Ajout d'une consultation 22
3.8 Diagramme de séquence : ms-rendez-vous 23
3.9 Diagramme de deploiement de MedSys 24
3.10 Modele Conceptuel de Donnees (MCD) 25
3.11 MCD : ms- rendez-vous 26

Liste des tableaux

2.1 Acteurs du systeme MedSys 11
2.2 Besoins non fonctionnels de MedSys 13

3.1 Microservices de MedSys 14
3.2 MLD — Tables principales 27
3.3 Dictionnaire de donnees — Entites principales 29

4.1 Outils et technologies utilises 31
4.2 Endpoints — ms-auth 31
4.3 Endpoints — ms-patient-personnel (extrait) 32
4.4 Difficultes techniques et solutions 34

Liste des abreviations

Abreviation	Signification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DME	Dossier Medical Electronique
DTO	Data Transfer Object
HTTP	HyperText Transfer Protocol
JPA	Java Persistence API
JWT	JSON Web Token
MCD	Modele Conceptuel de Donnees
MLD	Modele Logique de Donnees
MPD	Modele Physique de Donnees
ORM	Object-Relational Mapping
PFA	Projet de Fin d'Annee
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language

Chapitre 1

Introduction Generale

1.1 Contexte et Problematique

Le secteur de la sante connait aujourd'hui une transformation numerique profonde. Les etablissements hospitaliers, confrontes a une charge administrative croissante et a la necessite d'ameliorer la qualite des soins, se tournent de plus en plus vers des solutions logicielles integrees.

La gestion manuelle des dossiers patients, des consultations et des prescriptions presente de nombreux inconvenients : risque d'erreurs, perte d'informations, manque de tracabilite et difficulte d'acces aux donnees en temps reel. Ces limitations impactent directement la qualite des soins et l'efficacite du personnel medical.

Remarque

Problematique centrale : Comment concevoir et developper un systeme d'information hospitalier moderne, fiable, securise et facilement extensible, permettant de digitaliser et d'optimiser la gestion complete d'un hopital ?

1.2 Objectifs du Projet

MedSys a ete concu pour atteindre les objectifs suivants :

- Digitaliser l'ensemble du cycle de vie du dossier medical patient ;
- Permettre une gestion centralisee et securisee des donnees de sante ;
- Offrir une interface accessible sur ordinateur (web) et smartphone (mobile) ;
- Garantir la securite par une authentification robuste basee sur JWT ;
- Assurer la scalabilite via une architecture microservices ;
- Faciliter la communication entre patients et personnel medical.

1.3 Perimetre Fonctionnel

Le systeme MedSys couvre les domaines fonctionnels suivants :

1. **Authentification et gestion des roles** : connexion securisee, inscription patient, reinitialisation de mot de passe ;
2. **Gestion des patients** : creation, consultation, modification des fiches patients ;
3. **Dossier medical electronique** : consultations, antecedents, ordonnances, analyses, radiologies, hospitalisations ;
4. **Gestion du personnel medical** : medecins, specialites, services ;

5. **Messagerie interne** : communication patient/medecin ;
6. **Gestion des rendez-vous** : prise, modification, annulation.

1.4 Organisation du Rapport

Ce rapport est organise en cinq chapitres :

— **Chapitre 1** : Introduction generale ;
— **Chapitre 2** : Analyse et specification des besoins ;
— **Chapitre 3** : Conception UML et Merise ;
— **Chapitre 4** : Realisation et implementation ;
— **Chapitre 5** : Conclusion et perspectives.

Chapitre 2

Analyse et Specification des Besoins

2.1 Identification des Acteurs

L'analyse du systeme MedSys permet d'identifier quatre acteurs principaux :

Table 2.1 – Acteurs du systeme MedSys

Acteur	Description
Administrateur	Gere les comptes utilisateurs, cree les comptes medecins/-personnel, active ou desactive les comptes
Medecin	Consulte et enrichit les dossiers medicaux, redige des ordonnances, prescrit des analyses et radiologies, gere les hospitalisations
Patient	S'inscrit, consulte son dossier medical, communique avec son medecin, gere ses rendez-vous
Personnel hospitalier	Acces aux informations patients selon ses droits, gestion des services

2.2 Diagramme de Cas d'Utilisation Global

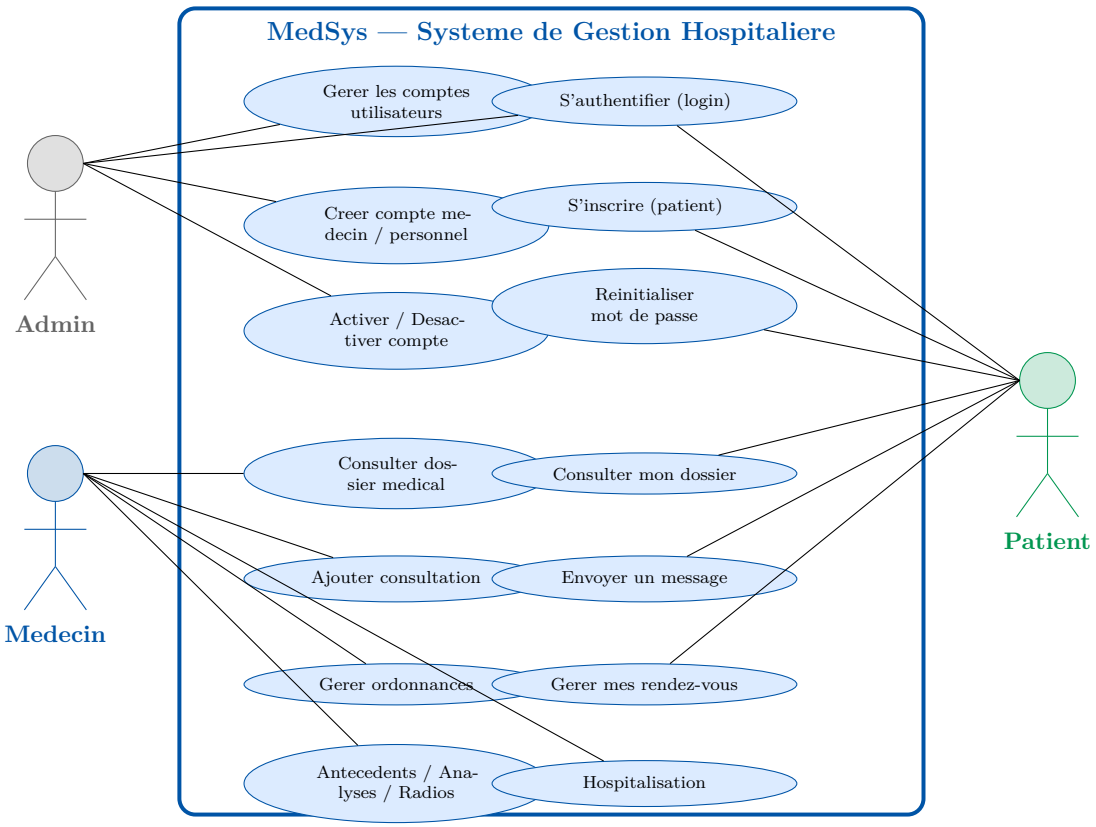


FIGURE 2.1 – Diagramme de cas d'utilisation global de MedSys

2.3 Besoins Fonctionnels

2.3.1 Module Authentification (ms-auth)

- **BF01** : Authentification via email/mot de passe, retour d'un token JWT ;
- **BF02** : Inscription patient avec creation automatique du dossier ;
- **BF03** : Reinitialisation du mot de passe par email ;
- **BF04** : Creation de comptes medecin/personnel par l'administrateur ;
- **BF05** : Activation/desactivation de comptes par l'administrateur ;
- **BF06** : Protection brute-force par rate limiting sur les connexions.

2.3.2 Module Patient et Dossier Medical (ms-patient-personnel)

- **BF07** : CRUD complet sur les fiches patients ;
- **BF08** : Chaque patient dispose d'un DME unique (numero DM-YYYY-XXXXX) ;
- **BF09** : Ajout de consultations avec signes vitaux (poids, tension, temperature) ;
- **BF10** : Enregistrement des antecedents medicaux avec niveau de severite ;
- **BF11** : Creation d'ordonnances avec lignes de medicaments ;
- **BF12** : Gestion des analyses de laboratoire (demande, resultats, statut) ;

— **BF13** : Gestion des examens de radiologie ;
— **BF14** : Gestion des hospitalisations (service, chambre, entree/sortie) ;
— **BF15** : Emission de certificats medicaux par les medecins ;
— **BF16** : Depot de documents (fichiers) dans le DME ;
— **BF17** : Messagerie patient/medecin ;
— **BF18** : Historique de toutes les actions sur le DME.

2.4 Besoins Non Fonctionnels

Table 2.2 – Besoins non fonctionnels de MedSys

Besoin	Description
Securite	JWT 24h, RBAC 4 roles, rate limiting, BCrypt pour les mots de passe
Scalabilite	Chaque microservice deployable et scalable independamment
Maintenabilite	Architecture en couches (Controller / Service / Repository)
Performance	Reponse API $<$ 500ms pour les operations CRUD standard
Accessibilite	Interface web responsive + application mobile iOS/Android
Interoperabilite	API REST documentee avec Swagger/OpenAPI

Chapitre 3

Conception

3.1 Architecture Generale

3.1.1 Choix Architectural : Microservices

MedSys adopte une **architecture microservices** ou chaque service est une application autonome, deployable independamment, communiquant via des APIs REST.

TABLE 3.1 – Microservices de MedSys

Service	Port	Responsabilite	Base de donnees
ms-auth	8082	Authentification, JWT, comptes	ms_auth_db
ms-patient-personnel	8081	Patients, DME, examens	ms_patient_db
medsys-web	5173	Interface web (React/Vite)	—
medsys-mobile	Expo	Application mobile (React Native)	—

3.1.2 Diagramme d'Architecture

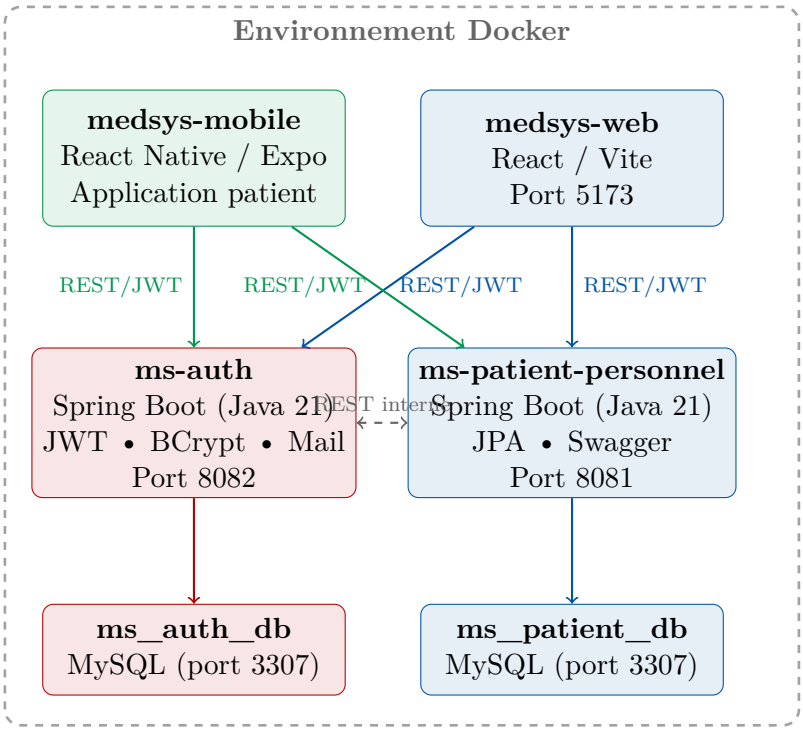


FIGURE 3.1 – Architecture generale de MedSys

3.2 Conception UML

3.2.1 Diagrammes des cas d'utilisation :

3.2.1.1 ms-rendez-vous

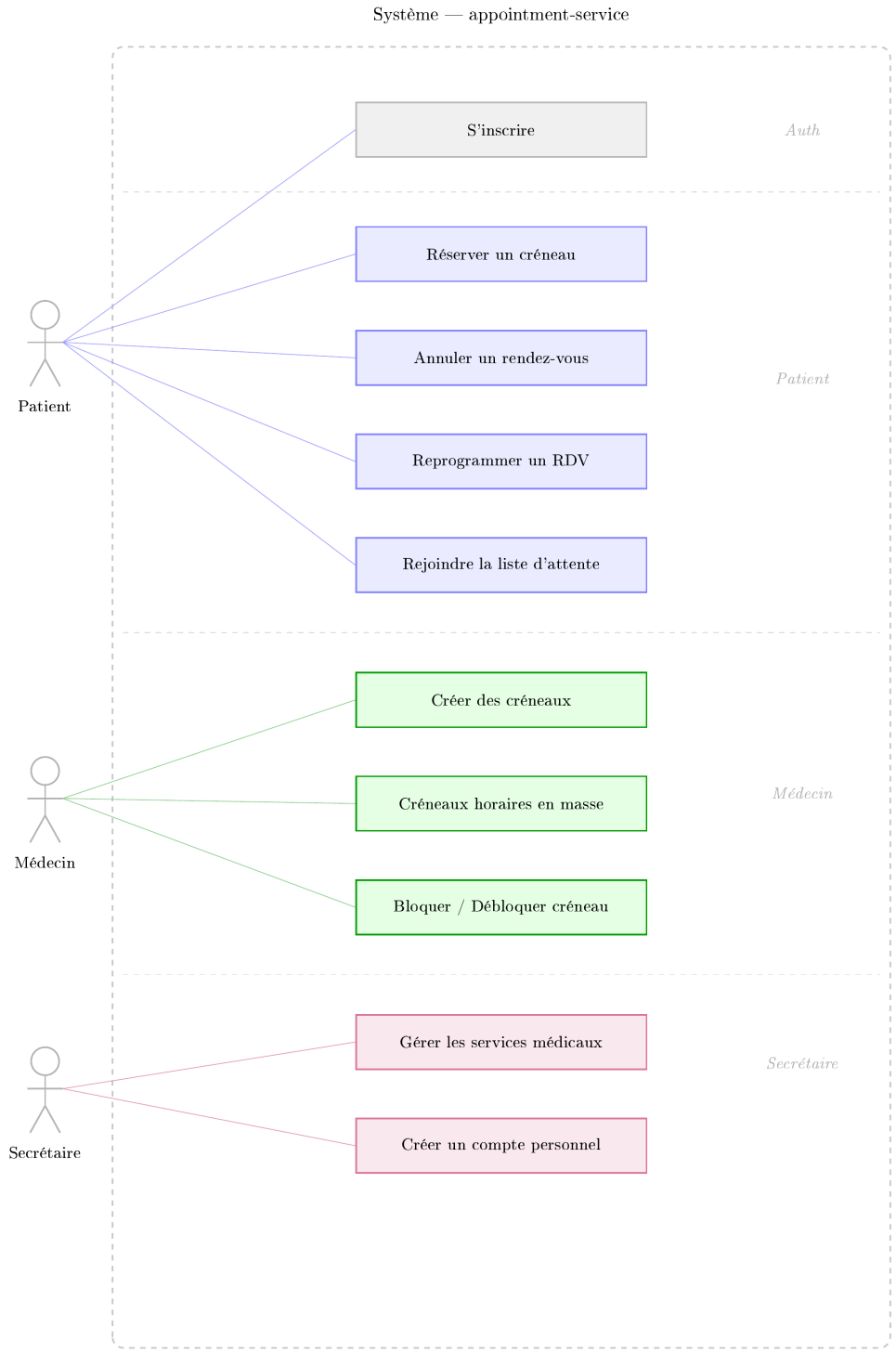


FIGURE 3.2 – Diagramme de use case : ms-rendez-vous

3.2.2 Diagrammes de classe

3.2.2.1 ms-auth

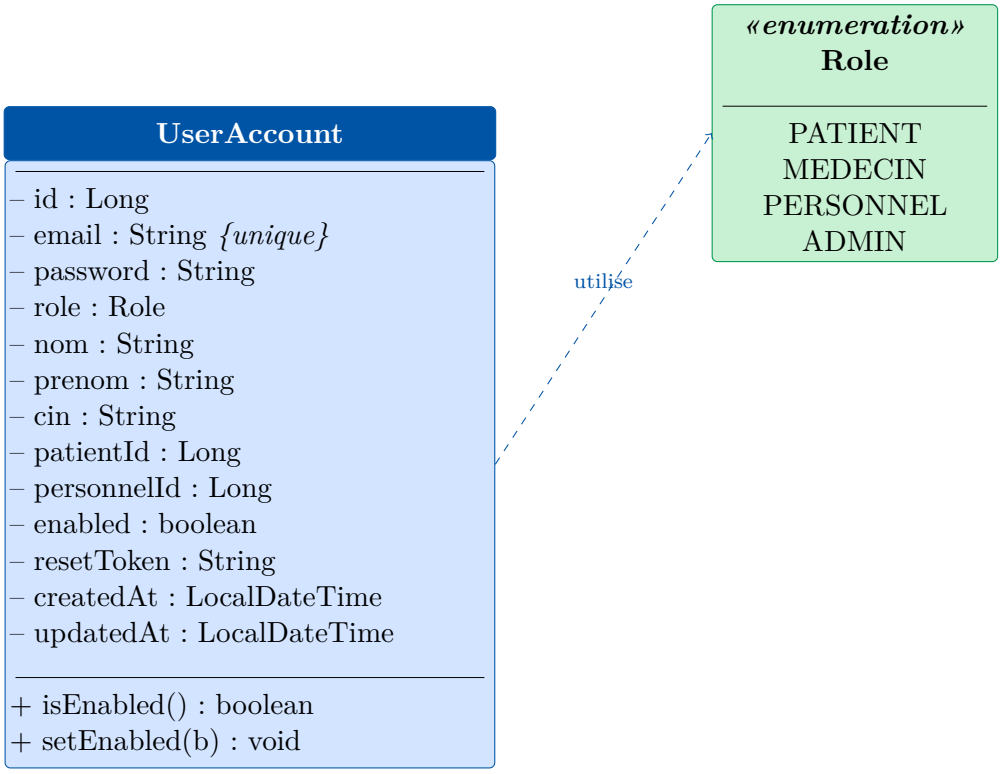


FIGURE 3.3 – Diagramme de classes — ms-auth

3.2.2.2 ms-patient-personnel

En raison de la richesse du modele de donnees, le diagramme de classes est presente en deux parties : les entites du dossier medical et les entites de personnel/reference.

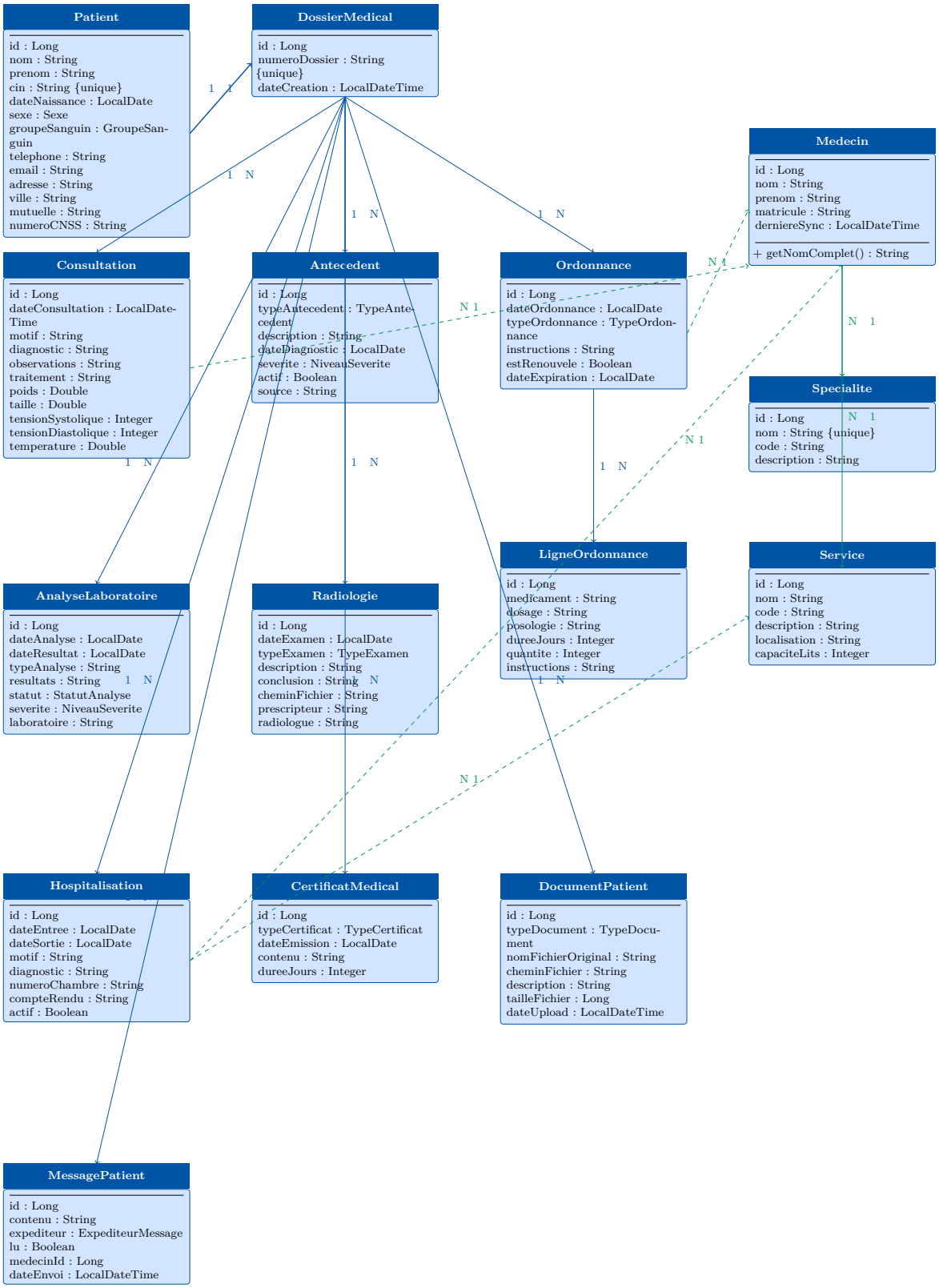


FIGURE 3.4 – Diagramme de classes — ms-patient-personnel

3.2.2.3 ms-rendez-vous

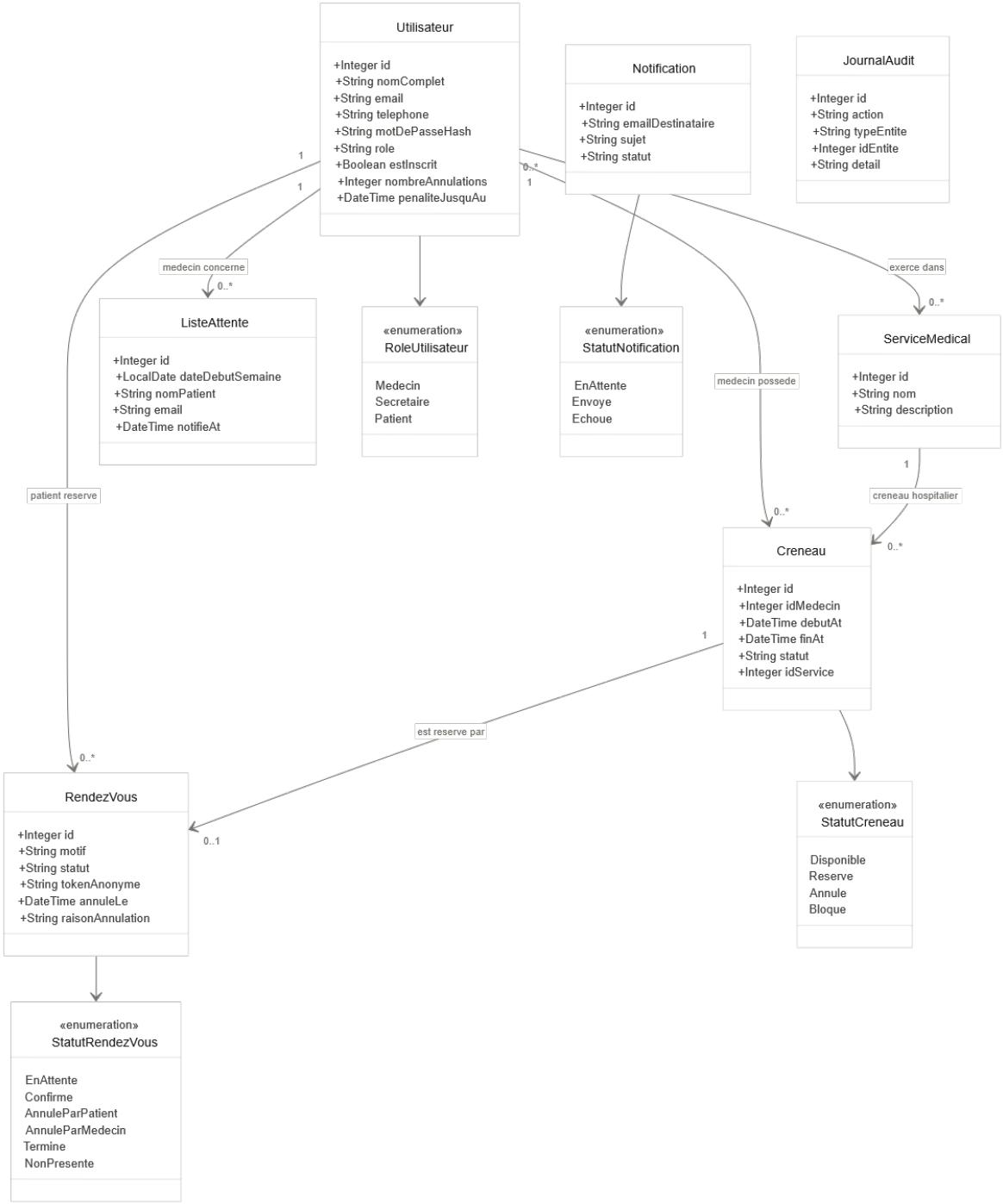
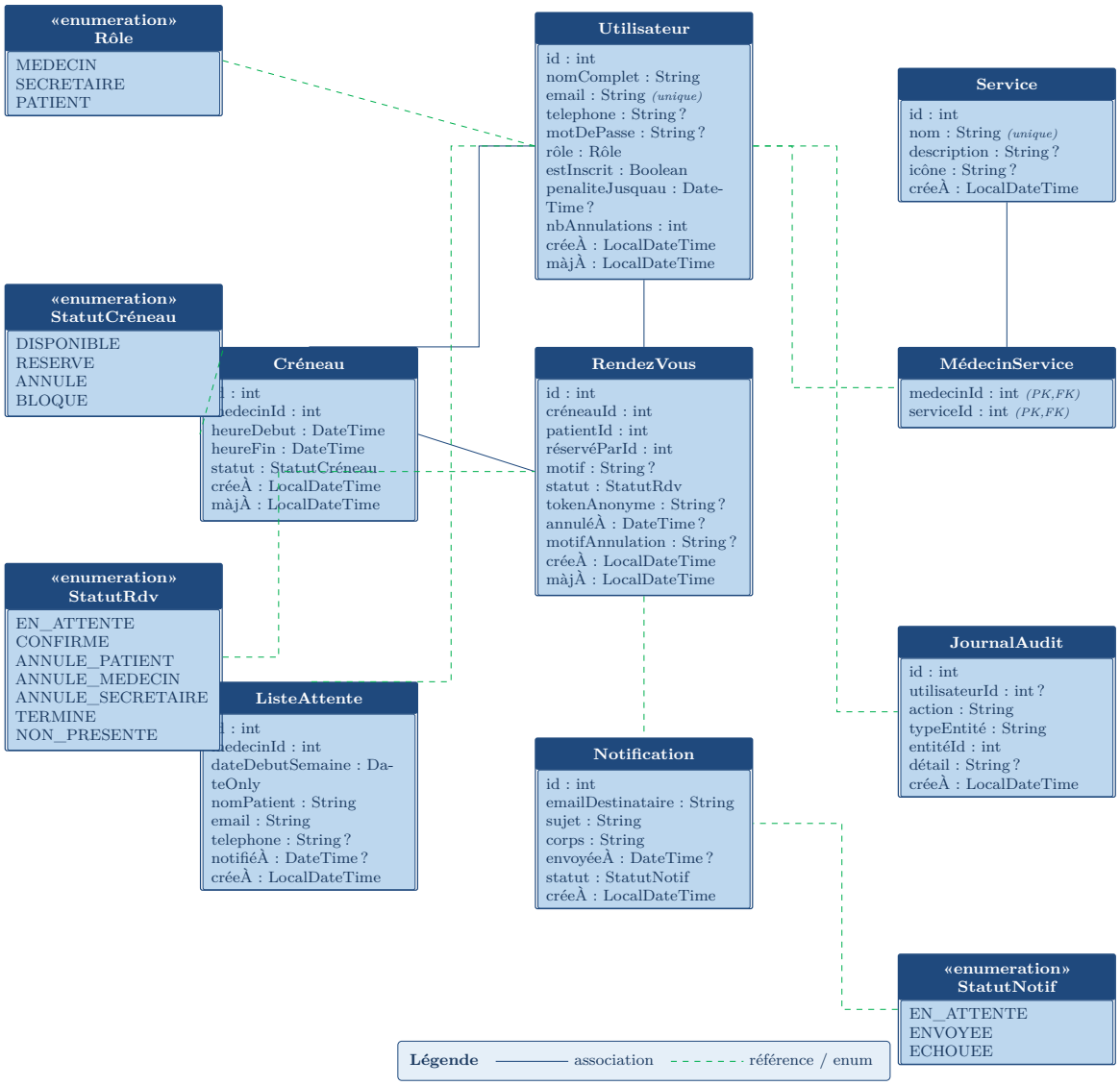


FIGURE 3.5 – Diagramme de classe : ms-rendez-vous


«enumeration»
Rôle
MEDECIN
SECRETAIRE
PATIENT
Utilisateur
id : int
nomComplet : String
email : String (unique)
telephone : String ?
motDePasse : String ?
rôle : Rôle
estInscrit : Boolean
penaliteJusquau : DateTime ?
nbAnnulations : int
créeÀ : LocalDateTime
màjÀ : LocalDateTime
Service
id : int
nom : String (unique)
description : String ?
icône : String ?
créeÀ : LocalDateTime
«enumeration»
StatutCréneau
DISPONIBLE
RESERVE
ANNULE
BLOQUE
Créneau
id : int
medecinId : int
heureDebut : DateTime
heureFin : DateTime
statut : StatutCréneau
créeÀ : LocalDateTime
màjÀ : LocalDateTime
RendezVous
id : int
créneauId : int
patientId : int
réservéParId : int
motif : String ?
statut : StatutRdv
tokenAnonyme : String ?
annuléÀ : DateTime ?
motifAnnulation : String ?
créeÀ : LocalDateTime
màjÀ : LocalDateTime
MédecinService
medecinId : int (PK,FK)
serviceId : int (PK,FK)
«enumeration»
StatutRdv
EN_ATTENTE
CONFIRME
ANNULE_PATIENT
ANNULE_MEDECIN
ANNULE_SECRETAIRE
TERMINE
NON_PRESENTE
JournalAudit
id : int
utilisateurId : int ?
action : String
typeEntité : String
entitéId : int
détail : String ?
créeÀ : LocalDateTime
ListeAttente
id : int
medecinId : int
dateDebutSemaine : DateOnly
nomPatient : String
email : String
telephone : String ?
notifiéÀ : DateTime ?
créeÀ : LocalDateTime
Notification
id : int
emailDestinataire : String
sujet : String
corps : String
envoyéeÀ : DateTime ?
statut : StatutNotif
créeÀ : LocalDateTime
«enumeration»
StatutNotif
EN_ATTENTE
ENVOYEE
ECHOUEE
Légende
association
référence / enum

3.2.3 Diagrammes de Sequence

3.2.3.1 Sequence : Authentification JWT

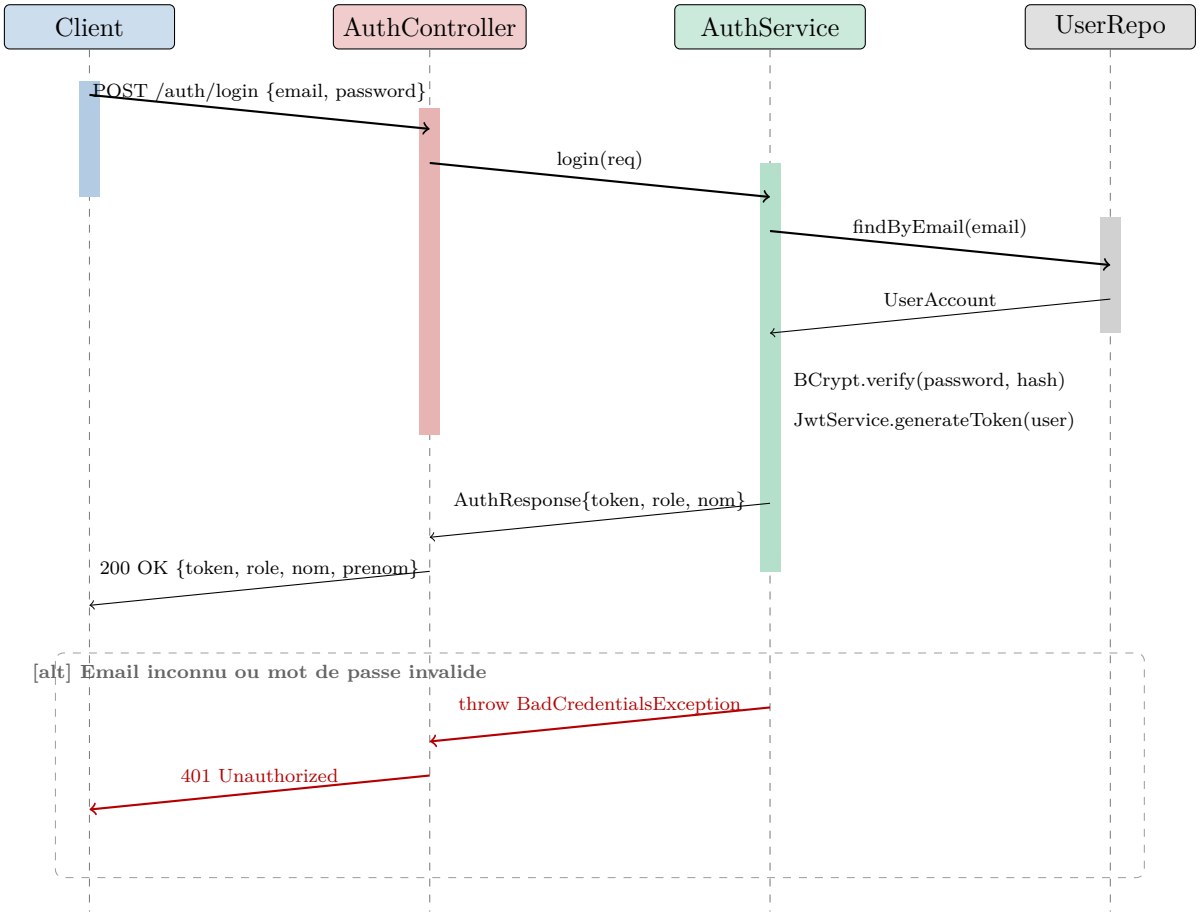


FIGURE 3.6 – Diagramme de sequence — Authentification JWT

3.2.3.2 Sequence : Ajout d'une Consultation

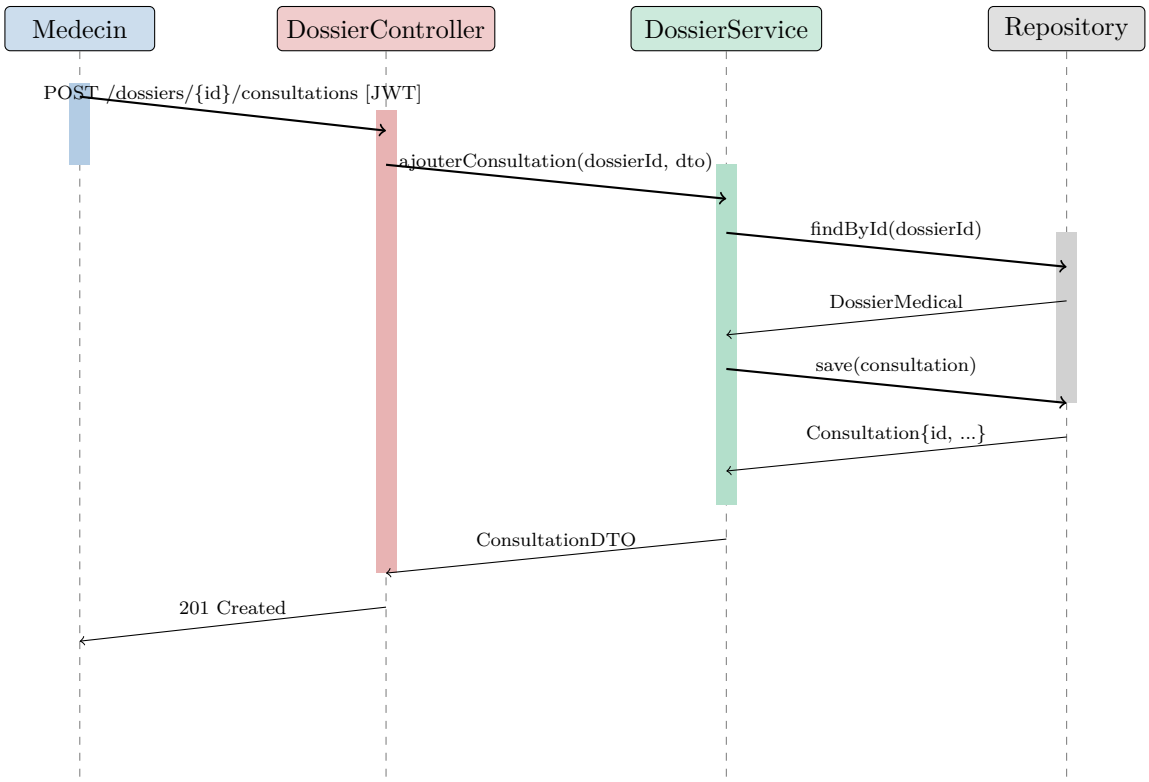


FIGURE 3.7 – Diagramme de sequence — Ajout d'une consultation

3.2.3.3 ms-rendez-vous

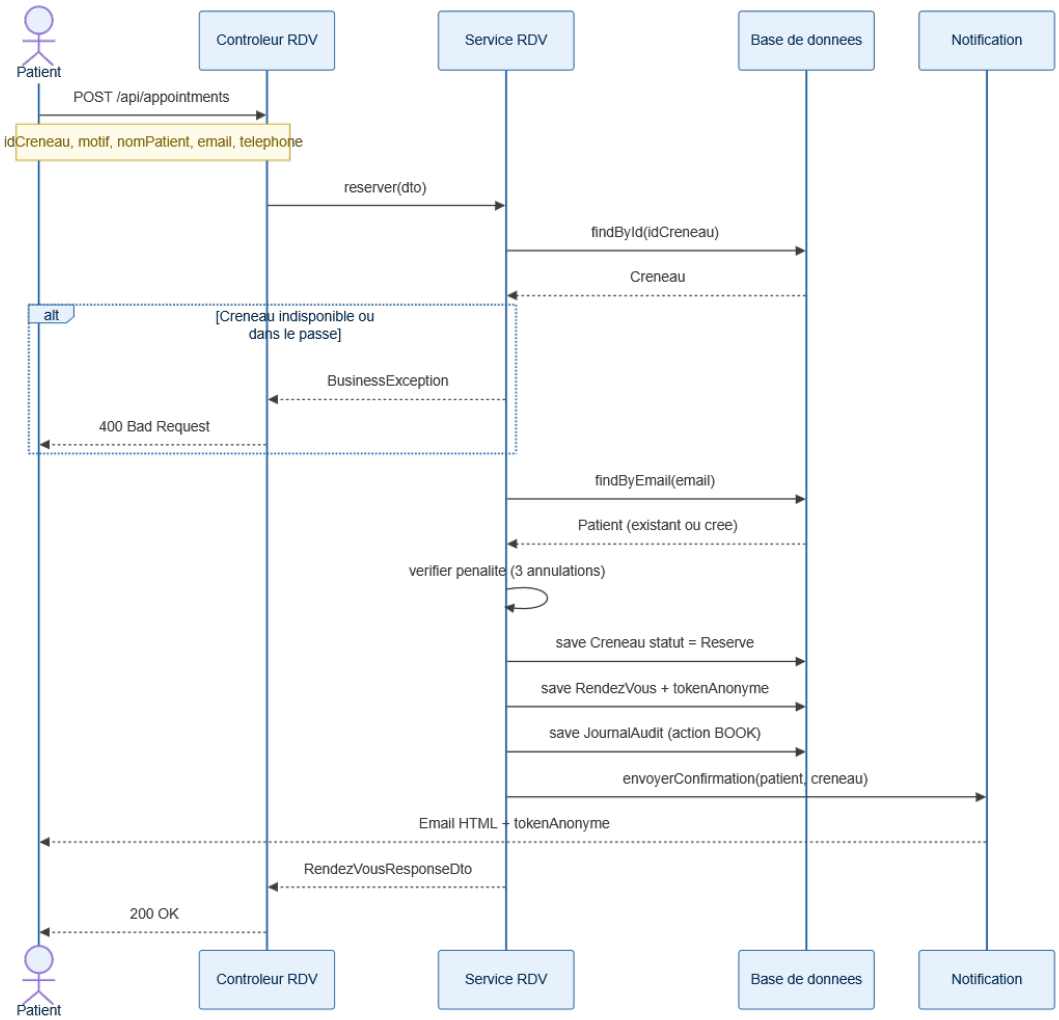


FIGURE 3.8 – Diagramme de séquence : ms-rendez-vous

3.2.4 Diagrammes de Deploiement

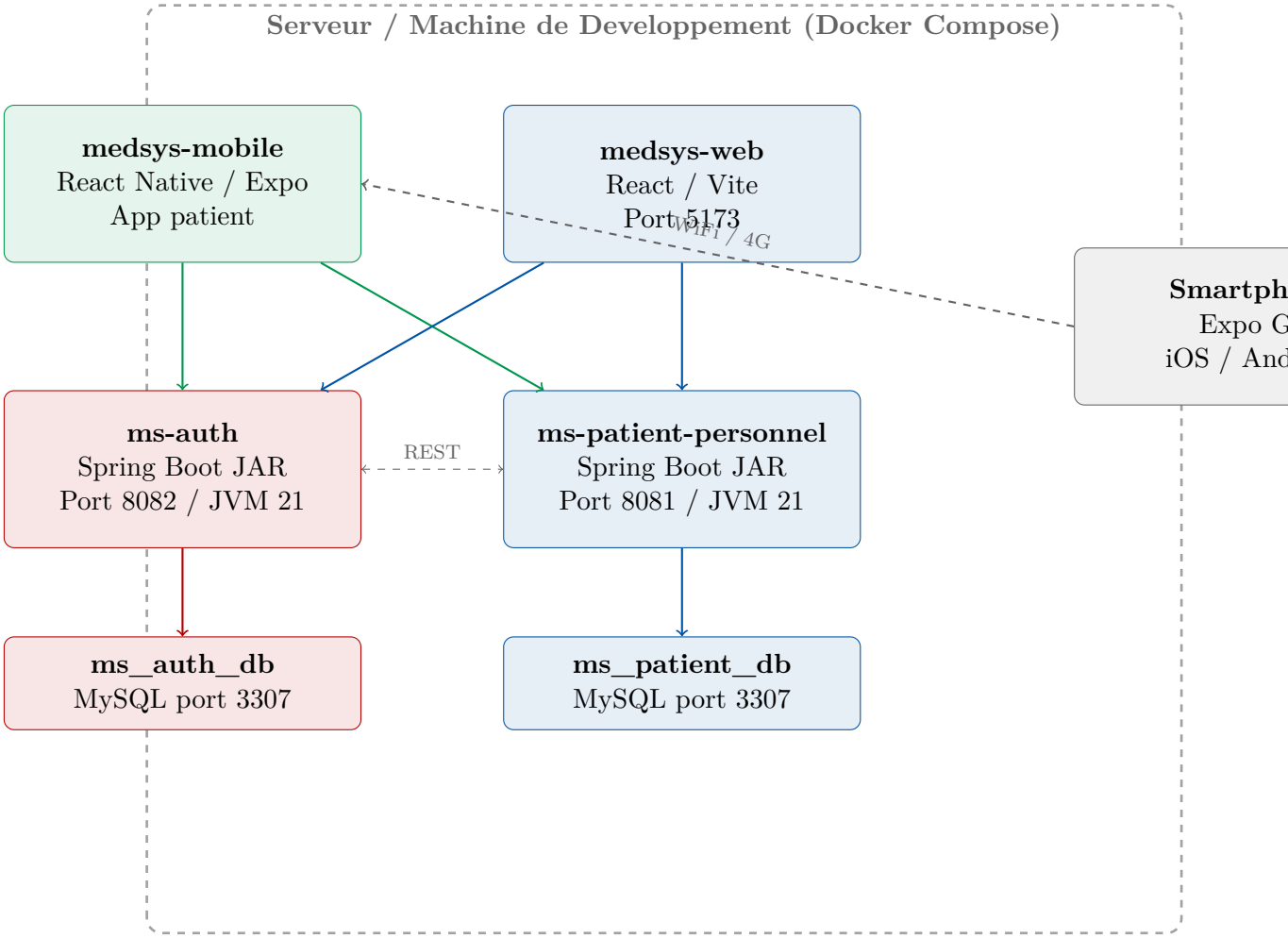


FIGURE 3.9 – Diagramme de deploiement de MedSys

3.3 Conception Merise

3.3.1 Modele Conceptuel de Donnees (MCD)

3.3.1.1 ————-

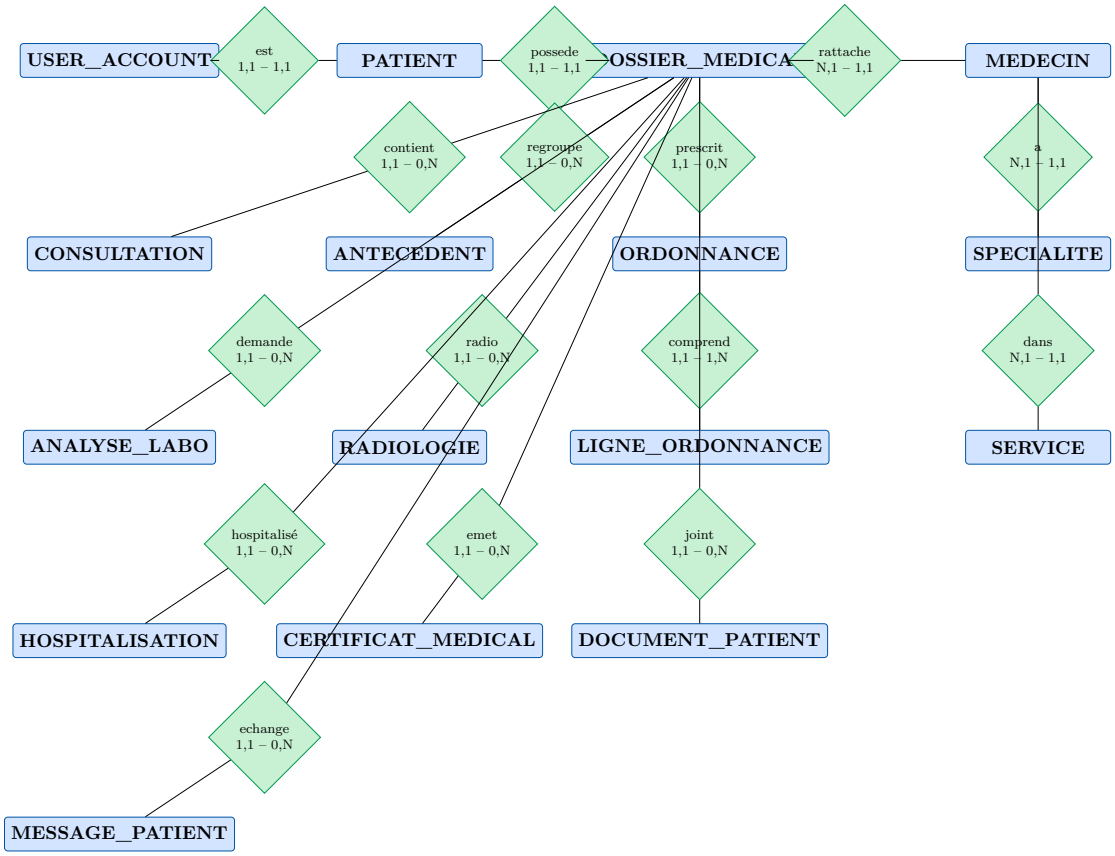


FIGURE 3.10 – Modele Conceptuel de Donnees (MCD)

3.3.1.2 ms- rendez-vous

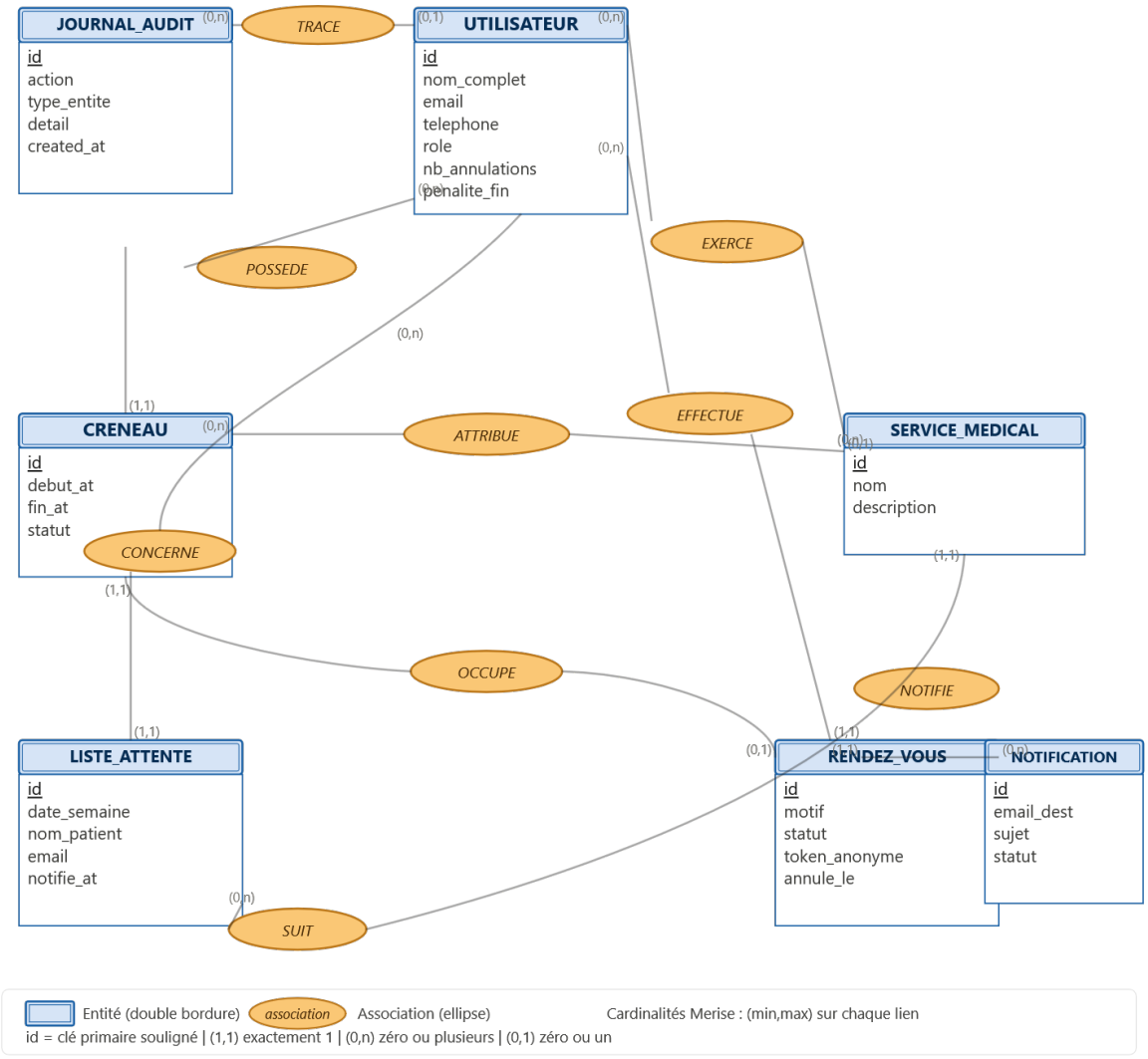


FIGURE 3.11 – MCD : ms- rendez-vous

3.3.2 Modele Logique de Donnees (MLD)

Le MLD traduit le MCD en tables relationnelles. Les cles primaires sont soulignees (id), les cles etrangeres sont precedees de #.

Table 3.2 – MLD — Tables principales

Table	Colonnes
user_accounts	id, email*, password, role, nom, prenom, cin, patient_id, personnel_id, enabled, reset_token, reset_token_expiry, created_at, updated_at
patients	id, nom, prenom, cin*, date_naissance, sexe, groupe_sanguin, telephone, email, adresse, ville, mutuelle, numero_cnss, #dossier_medical_id, created_at, updated_at
dossiers_medicaux	id, numero_dossier*, date_creation
consultations	id, date_consultation, motif, diagnostic, observations, traitement, poids, taille, tension_systolique, tension_diastolique, temperature, #dossier_id, #medecin_id
antecedents	id, type_antecedent, description, date_diagnostic, severite, actif, source, #dossier_id
ordonnances	id, date_ordonnance, type_ordonnance, instructions, est_renouvele, date_expiration, #dossier_id, #medecin_id
lignes_ordonnance	id, medicament, dosage, posologie, duree_jours, quantite, instructions, #ordonnance_id
analyses_laboratoire	id, date_analyse, date_resultat, type_analyse, resultats, valeur_reference, statut, severite, laboratoire, prescripteur, chemin_fichier, #dossier_id
radiologies	id, date_examen, type_examen, description, conclusion, chemin_fichier, prescripteur, radiologue, #dossier_id
hospitalisations	id, date_entree, date_sortie, motif, diagnostic, numero_chambre, compte_rendu, actif, #dossier_id, #service_id, #medecin_responsable_id
certificats_medicaux	id, type_certificat, date_emission, contenu, duree_jours, #dossier_id, #medecin_id
documents_patient	id, type_document, nom_fichier_original, chemin_fichier, description, taille_fichier, content_type, date_upload, #dossier_id
messages_patient	id, contenu, expediteur, lu, medecin_id, medecin_nom, date_envoi, #dossier_id
medecins_ref	id, nom, prenom, matricule, derniere_sync, #specialite_id, #service_id
specialites	id, nom*, code*, description
services	id, nom, code, description, localisation, capacite_lits
etats_actuels	id, symptomes, traitement_en_cours, allergies_actives, poids_actuel, taille_actuelle, date_mise_a_jour
historique_medical	id, action, details, utilisateur, date_action

** = UNIQUE # = Cle etrangere id = Cle primaire*

3.3.3 Modele Physique de Donnees (MPD)

```sql
-- Base de donnees : ms_patient_db

CREATE TABLE dossiers_medicaux (
  id              BIGINT AUTO_INCREMENT PRIMARY KEY,
  numero_dossier VARCHAR(30)  NOT NULL UNIQUE,
  date_creation  DATETIME     DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE patients (
  id                  BIGINT AUTO_INCREMENT PRIMARY KEY,
  nom                 VARCHAR(100) NOT NULL,
  prenom              VARCHAR(100) NOT NULL,
  cin                 VARCHAR(20)  NOT NULL UNIQUE,
  date_naissance      DATE         NOT NULL,
  sexe                ENUM('MASCULIN','FEMININ'),
  groupe_sanguin      ENUM('A_POS','A_NEG','B_POS','B_NEG',
                           'AB_POS','AB_NEG','O_POS','O_NEG'),
  telephone           VARCHAR(20),
  email               VARCHAR(150),
  adresse             TEXT,
  ville               VARCHAR(100),
  mutuelle            VARCHAR(100),
  numero_cnss         VARCHAR(50),
  dossier_medical_id BIGINT UNIQUE,
  created_at          DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at          DATETIME DEFAULT CURRENT_TIMESTAMP
                      ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (dossier_medical_id)
    REFERENCES dossiers_medicaux(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE consultations (
  id                   BIGINT AUTO_INCREMENT PRIMARY KEY,
  date_consultation    DATETIME NOT NULL,
  motif                VARCHAR(255),
  diagnostic           TEXT,
  observations         TEXT,
  traitement           TEXT,
  poids                DOUBLE,
  taille               DOUBLE,
  tension_systolique   INT,
  tension_diastolique  INT,
  temperature          DOUBLE,
  dossier_id           BIGINT NOT NULL,
  medecin_id           BIGINT,
  FOREIGN KEY (dossier_id) REFERENCES dossiers_medicaux(id),
  FOREIGN KEY (medecin_id) REFERENCES medecins_ref(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE ordonnances (
  id                BIGINT AUTO_INCREMENT PRIMARY KEY,
  date_ordonnance   DATE,
  type_ordonnance   ENUM('CLASSIQUE','STUPEFIANT','BIZONE'),
  instructions      TEXT,
  est_renouvele     TINYINT(1) DEFAULT 0,
  date_expiration   DATE,
```

```sql
  dossier_id        BIGINT,
  medecin_id        BIGINT,
  FOREIGN KEY (dossier_id) REFERENCES dossiers_medicaux(id),
  FOREIGN KEY (medecin_id) REFERENCES medecins_ref(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Base de donnees : ms_auth_db

CREATE TABLE user_accounts (
  id                  BIGINT AUTO_INCREMENT PRIMARY KEY,
  email               VARCHAR(150) NOT NULL UNIQUE,
  password            VARCHAR(255) NOT NULL,
  role                ENUM('PATIENT','MEDECIN',
                           'PERSONNEL','ADMIN') NOT NULL,
  nom                 VARCHAR(100) NOT NULL,
  prenom              VARCHAR(100) NOT NULL,
  cin                 VARCHAR(20),
  patient_id          BIGINT,
  personnel_id        BIGINT,
  enabled             TINYINT(1)   DEFAULT 1,
  reset_token         VARCHAR(255),
  reset_token_expiry  DATETIME,
  created_at          DATETIME     DEFAULT CURRENT_TIMESTAMP,
  updated_at          DATETIME     DEFAULT CURRENT_TIMESTAMP
                      ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Listing 3.1 – MPD — Tables principales (MySQL)

3.3.4 Dictionnaire de Donnees

Table 3.3 – Dictionnaire de donnees — Entites principales

Attribut	Type SQL	Contraintes	Description
TABLE : patients			
id	BIGINT	PK, AUTO	Identifiant unique
nom	VARCHAR(100)	NOT NULL	Nom de famille
prenom	VARCHAR(100)	NOT NULL	Prenom
cin	VARCHAR(20)	NOT NULL, UNIQUE	Carte nationale
date_naissance	DATE	NOT NULL	Date de naissance
sexe	ENUM	–	MASCULIN / FEMININ
groupe_sanguin	ENUM	–	Groupe ABO + Rh
telephone	VARCHAR(20)	–	Telephone
email	VARCHAR(150)	–	Email
mutuelle	VARCHAR(100)	–	Organisme assurance
numero_cnss	VARCHAR(50)	–	Numero CNSS
TABLE : dossiers_medicaux			
id	BIGINT	PK, AUTO	Identifiant unique
numero_dossier	VARCHAR(30)	NOT NULL, UNIQUE	Format DM-YYYY-XXXXX
date_creation	DATETIME	AUTO	Creation automatique
TABLE : consultations			

Table 3.3 (suite)

Attribut	**Type SQL**	**Contraintes**	**Description**
id	BIGINT	PK, AUTO	Identifiant unique
date_consultation	DATETIME	NOT NULL	Date et heure
motif	VARCHAR(255)	–	Motif de consultation
diagnostic	TEXT	–	Diagnostic medecin
poids	DOUBLE	–	Poids en kg
taille	DOUBLE	–	Taille en cm
tension_systolique	INT	–	Tension en mmHg
temperature	DOUBLE	–	Temperature en degres C
TABLE : ordonnances			
id	BIGINT	PK, AUTO	Identifiant unique
date_ordonnance	DATE	–	Date d'emission
type_ordonnance	ENUM	–	Type ordonnance
est_renouvele	BOOLEAN	DEFAULT FALSE	Renouvelable
date_expiration	DATE	–	Date expiration
TABLE : user_accounts			
id	BIGINT	PK, AUTO	Identifiant unique
email	VARCHAR(150)	NOT NULL, UNIQUE	Email connexion
password	VARCHAR(255)	NOT NULL	Hash BCrypt
role	ENUM	NOT NULL	Role RBAC
enabled	BOOLEAN	DEFAULT TRUE	Compte actif
reset_token	VARCHAR(255)	–	Token reinit. MDP

Chapitre 4

Realisation et Implementation

4.1 Environnement de Developpement

Table 4.1 – Outils et technologies utilises

Categorie	Outil / Technologie	Version
Langage Backend	Java	21
Framework Backend	Spring Boot	3.x
Persistence	Spring Data JPA / Hibernate	–
Securite	Spring Security + JWT (JJWT)	–
Build	Maven	3.9
SGBD	MySQL	8.x
Frontend Web	React + Vite + TypeScript	–
Mobile	React Native + Expo	–
Conteneurisation	Docker + Docker Compose	–
Documentation API	Swagger / OpenAPI 3	–
IDE	IntelliJ IDEA	–
Gestion de versions	Git + GitHub	–

4.2 API REST des Microservices

4.2.1 ms-auth (port 8082)

Table 4.2 – Endpoints — ms-auth

Methode	Endpoint	Description	Role
POST	/api/v1/auth/login	Connexion, retourne JWT	Public
POST	/api/v1/auth/register	Inscription patient	Public
POST	/api/v1/auth/forgot-password	Demande reinit. MDP	Public
POST	/api/v1/auth/reset-password	Reinitialiser MDP	Public
POST	/api/v1/admin/personnel	Creer compte medecin	ADMIN
GET	/api/v1/admin/users	Lister tous les comptes	ADMIN
GET	/api/v1/admin/users/role/{role}	Filtrer par role	ADMIN
PUT	/api/v1/admin/users/{id}/toggle	Activer/Desactiver	ADMIN
DELETE	/api/v1/admin/users/{id}	Supprimer compte	ADMIN

4.2.2 ms-patient-personnel (port 8081)

Table 4.3 – Endpoints — ms-patient-personnel (extrait)

Methode	Endpoint	Description	Role
GET	/api/v1/patients	Lister patients	MEDECIN, ADMIN
POST	/api/v1/patients	Creer patient	MEDECIN
GET	/api/v1/patients/{id}	Details patient	Tous
PUT	/api/v1/patients/{id}	Modifier patient	MEDECIN
DELETE	/api/v1/patients/{id}	Supprimer patient	ADMIN
GET	/api/v1/dossiers/{id}	Consulter le DME	Tous
POST	/api/v1/dossiers/{id}/consultations	Ajouter consultation	MEDECIN
POST	/api/v1/dossiers/{id}/ordonnances	Creer ordonnance	MEDECIN
POST	/api/v1/dossiers/{id}/antecedents	Ajouter antecedent	MEDECIN
POST	/api/v1/dossiers/{id}/analyses	Demander analyse	MEDECIN
POST	/api/v1/dossiers/{id}/radiologies	Prescrire radio	MEDECIN
POST	/api/v1/dossiers/{id}/hospitalisations	Hospitaliser	MEDECIN
POST	/api/v1/dossiers/{id}/certificats	Emettre certificat	MEDECIN
GET	/api/v1/patient/dossier	Mon dossier (patient)	PATIENT
POST	/api/v1/patient/messages	Envoyer message	PATIENT
GET	/api/v1/patient/rdv	Mes rendez-vous	PATIENT

4.3 Securite : JWT et RBAC

La securite de MedSys repose sur deux mecanismes :

— **JWT** : token signe genere a chaque connexion reussie, valide 24h, contenant le role et les informations de l'utilisateur ;

— **RBAC** : 4 roles (PATIENT, MEDECIN, PERSONNEL, ADMIN) controlant l'acces a chaque endpoint via `@PreAuthorize`.

```java
public String generateToken(UserAccount user) {
    return Jwts.builder()
        .setSubject(user.getEmail())
        .claim("role", user.getRole().name())
        .claim("nom", user.getNom())
        .claim("prenom", user.getPrenom())
        .claim("patientId", user.getPatientId())
        .setIssuedAt(new Date())
        .setExpiration(
          new Date(System.currentTimeMillis() + 86400000L))
        .signWith(getSignKey(), SignatureAlgorithm.HS256)
        .compact();
}
```

Listing 4.1 – Generation du token JWT

4.4 Numerotation Automatique des Dossiers

Chaque dossier medical est identifie par un numero unique au format **DM-YYYY-XXXXX**, genere automatiquement via un callback JPA :

```java
@PrePersist
public void generateNumeroDossier() {
    if (numeroDossier == null) {
        numeroDossier = "DM-"
            + LocalDate.now().getYear() + "-"
            + String.format("%05d",
                System.currentTimeMillis() % 100000);
    }
}
```

Listing 4.2 – Generation du numero de dossier (@PrePersist)

4.5 Interfaces Utilisateur

4.5.1 Application Web (medsys-web)

L'interface web est developpee avec React et Vite (port 5173). Elle propose deux espaces :

— **Espace Personnel** : gestion complete des patients et des dossiers medicaux ;

— **Espace Patient** : inscription multi-etapes (6 etapes), consultation du dossier, messagerie.

4.5.2 Application Mobile (medsys-mobile)

L'application mobile est developpee avec React Native et Expo. Elle permet aux patients d'acceder a leurs informations medicales depuis smartphone (iOS et Android) via Expo Go.

4.6 Difficultes Rencontrees et Solutions

Table 4.4 – Difficultes techniques et solutions

Difficulte	**Solution adoptee**
Donnees medecin necessaires dans ms-patient sans coupler les services	Entite locale `Medecin` synchronisee depuis ms-personnel
Gestion des CORS entre frontend et backends	Configuration centralisee Spring Security
IP variable en mobile (WiFi/4G)	Parametre dynamique dans api.js
Numerotation unique des dossiers	Callback `@PrePersist` avec format DM-YYYY-XXXXX

Chapitre 5

Conclusion et Perspectives

5.1 Bilan du Projet

MedSys est un systeme de gestion hospitaliere complet qui repond aux objectifs fixes en debut de projet. Nous avons concu et implemente une architecture microservices moderne, securisee et extensible, couvrant les principaux besoins d'un etablissement hospitalier.

Les principaux acquis de ce projet sont :

— Maitrise de l'architecture microservices avec Spring Boot ;

— Implementation d'une authentification securisee JWT + RBAC ;

— Developpement d'interfaces modernes (React/Vite, React Native) ;

— Containerisation avec Docker ;

— Documentation API avec Swagger/OpenAPI.

5.2 Perspectives d'Amelioration

1. **API Gateway** : Spring Cloud Gateway pour centraliser le routage ;
2. **Service Discovery** : Eureka pour la decouverte automatique des microservices ;
3. **Intelligence Artificielle** : aide a la decision medicale, detection d'interactions medicamenteuses ;
4. **Temps reel** : WebSocket pour les alertes medicales ;
5. **Cloud** : deploiement AWS/Azure avec Kubernetes ;
6. **Telemedecine** : consultation video integree.

5.3 Conclusion

Ce Projet de Fin d'Annee nous a permis de mettre en pratique l'ensemble des connaissances acquises durant notre cursus de Genie Informatique a HEI Oujda. La complexite du domaine medical, la rigueur exigee par l'architecture microservices et les defis de l'integration frontend/mobile ont ete autant de defis formateurs.

MedSys constitue une base solide pour un systeme d'information hospitalier reel, et les technologies choisies — Spring Boot, React, React Native et Docker — offrent l'ecosysteme ideal pour son evolution future.

Bibliographie

[1] Spring Team, *Spring Boot Reference Documentation*, v3.x, https://docs.spring.io/spring-boot/docs/current/reference/html/

[2] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.

[3] S. Newman, *Building Microservices*, 2e ed., O'Reilly Media, 2021.

[4] Meta, *React Documentation*, https://react.dev/

[5] Meta, *React Native Documentation*, https://reactnative.dev/

[6] Docker Inc., *Docker Documentation*, https://docs.docker.com/

[7] OMG, *UML 2.5 Specification*, https://www.omg.org/spec/UML/2.5/

[8] H. Tardieu, A. Rochfeld, R. Colletti, *La Methode Merise*, Les Editions d'Organisation, 1985.